

devreport: Project Reports Whose Numbers Come from Code

Gaurav Choudhury

October 2026

Abstract

Student builders often have to write up a hackathon, coursework or side project when they would rather keep building, and a chatbot asked to do the write-up will invent a results section. devreport takes a project as loose files, a `.zip` or a public GitHub link and produces a LaTeX research-paper report or a Beamer slide deck, compiled to PDF while the user watches. Plain, tested code in `collect.js` analyses the inputs first and writes every number into CSV files and `facts.json`; an agent then writes only the prose and the LaTeX, and every chart is a pgfplots plot that reads one of those files. The agent runs in a sandbox of tools that can only write inside the job folder and compile with `tectonic`. I built it solo for the FirstCommit hackathon. The test suite runs 18 tests, all passing, in 0.88s. On my MacBook a warm compile of a 10-slide deck takes between 1.17s and 1.34s across the seven slide themes, and 24 local test runs reached their first PDF in between 0.6 and 12.69 minutes.

1 Introduction

Writing up a project is a chore that comes at the end, when the deadline is close. Hackathon judges, course staff and readers of a portfolio want to see what was built, how it works and evidence that it works. A general chatbot can produce a document of the right shape, but it will happily fill the results section with numbers that were never measured. devreport is aimed at one narrow group of users: student builders turning their logs, notes, git history and screenshots into a report or slides, rather than turning any idea into any document.

The central idea is to keep the numbers away from the model. A deterministic collector parses the uploaded material into CSV files and a `facts.json` summary. The language model reads those files, the notes and key code files, and writes the prose around them. Charts are pgfplots commands that point at the collector's CSV files, so a wrong chart would need a bug in tested code rather than a hallucination. The rest of this paper describes the pipeline (section 2), the main implementation decisions (section 3), the measurements I made (section 4) and what I learned while building it (section 5).

2 System overview

A job moves through a fixed pipeline. The user uploads files, a `.zip` or a GitHub link, chooses a report or slides, a length, and for slides a theme or an uploaded PowerPoint template (Figure 1). The steps are:

1. `server.js`, a Node standard-library HTTP server, unpacks a zip or clones the repository and saves the upload into `jobs/<id>/input/`.
2. `collect.js` turns git history, logs, tables, notes and images into CSV files under `data/` and a `facts.json` summary. If a `.pptx` template was uploaded, `pptx.js` extracts its colours, fonts, media, layout boxes and thumbnail into `template/`.

3. The agent reads `facts.json`, the notes and key code files. If it cannot tell what the project is or who it is for, it writes up to three questions to `questions.json` and stops; the user answers or skips and the same session resumes.
4. The agent writes `main.tex`, compiles it with `tectonic`, reads the errors, fixes them and recompiles.
5. The finished `main.pdf` is shown in the viewer, with the `.tex` available to download.

The screenshot shows the 'devreport' web interface. At the top, it says 'devreport v0.1 · LOCAL'. The main heading is 'Your messy repo, typeset.' in a serif font. To the right, a paragraph explains: 'Drop in dev logs, notes, screenshots, a zip, or a GitHub link. devreport counts the commits and charts the real numbers. The agent then writes the LaTeX, compiles it, fixes its own errors, and gives you a PDF report or slide deck.' Below this, there are three sections:

- 01 Source**: A drop zone for files or a zip, with a text input for a GitHub link (e.g., 'github.com/owenr/tepo').
- 03 Layout**: A button labeled 'Generate report'.
- 02 Format**: Two buttons for 'Report' (Article, A4 pages) and 'Slides' (Beamer deck, 16:9). Below these is a 'PAGES' slider set to 4 pages.

Figure 1: The upload form: a drop zone for files or a zip, a field for a GitHub link, the choice between a report and slides, and the length slider.

Progress streams to the browser over server-sent events as a live log grouped by phase (Uploading, Analysing, Reading, Writing, Compiling, Polishing, Done). The viewer is built on pdf.js and offers slide thumbnails, a present mode, a theme switcher and notes pinned to a page (Figure 2). A finished job can be revised with page notes or general notes, in which case the agent edits the existing LaTeX and earlier versions are kept as backups, and more material can be attached later so that the collector re-runs and the agent works it in.

3 Implementation

3.1 Keeping the numbers honest

The collector does all analysis in plain code covered by `test.js`: milestones from git history with vendored and generated code left out, likely entry points, recurring log errors grouped by replacing numbers, hex values and IDs with placeholders, and the column types of the user's own CSV or JSON tables. Every chart in the output is a `pgfplots \addplot table` reading a file the collector wrote, and stat numbers in the prose are copied from `facts.json`. Commit counts and lines of code are still computed but are not offered as charts, since they do not show that a project works. Sections such as lessons learned and future work are left out when the uploaded files say nothing about them.

3.2 One source file for every theme

A single `main.tex` serves every look. It loads its theme through one line, `\input{theme.tex}`, and each theme is a small adapter `.sty` on a shared core, `devreport-core.sty`, which defines

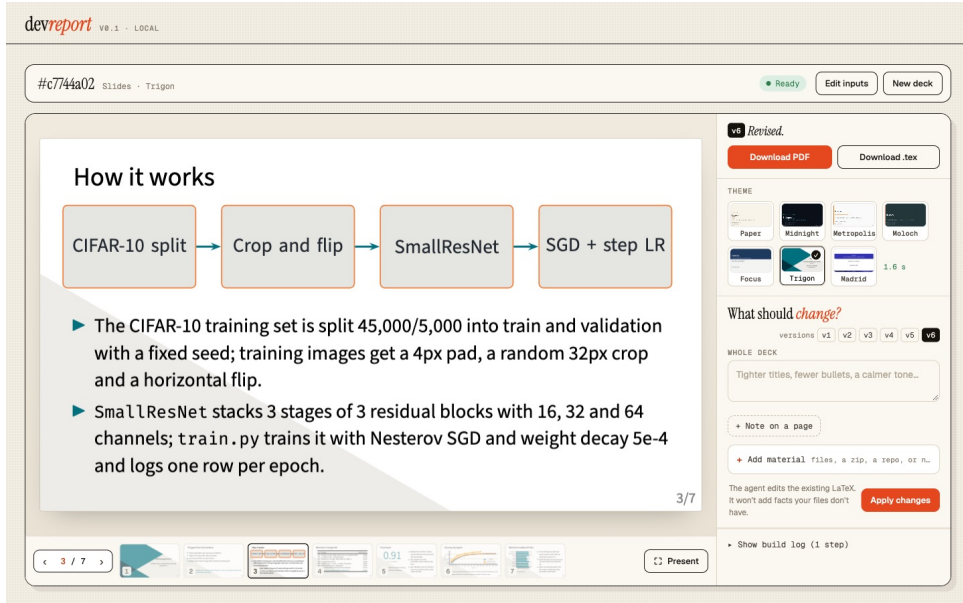


Figure 2: The viewer showing a finished seven-slide deck in the Trigon theme, with page thumbnails, the theme switcher, the version history and the revision panel.

the same `\dr*` colours, chart styles, stat tiles and layout macros. Reports use one standard article layout. Slides come in seven themes (Paper, Midnight, Metropolis, Moloch, Focus, Trigon and Madrid), or a custom theme built from an uploaded `.pptx`. Switching theme rewrites `theme.tex` and recompiles without calling the model, and every compiled theme is cached. The skill bans raw font sizes, so the agent uses named sizes such as `\drTitle`, `\drBody` and `\drStat`, and each theme sets those sizes to suit its own frame.

3.3 A sandbox made of tools

The default `cli` engine runs Claude Code headless (`claude -p`) with only these tools allowed: `Read`, `Write(./**)`, `Edit(./**)`, `Bash(tectonic:*)` and `Skill`, with the working directory set to the job folder. There is no general shell, and writes stay inside the job folder. The `ai` engine in `agent.js` is my own agent loop on the Vercel AI SDK, which lets the model be swapped for OpenAI, Anthropic, Google, OpenRouter or any OpenAI-compatible server, local Ollama included. It exposes exactly four tools, `read_file`, `write_file`, `edit_file` and `compile`, and every path goes through one check that refuses anything outside the job folder, any symlink and the server’s own files.

Intake has its own defences. Uploads are capped at 50 MB; the unpacked size of a zip is summed from `unzip -l` and refused over 200 MB, entries that resolve outside `input/` are rejected, and symlinks are deleted afterwards. GitHub links are normalised, matched against a strict owner/repo pattern and cloned with `execFile`, no shell, `GIT_TERMINAL_PROMPT=0` and a timeout. The skill tells the agent that everything under `input/`, `data/` and `template/` is untrusted data, and the tool sandbox limits the damage if that rule fails.

3.4 Timeline

The project was built in a single day. Table 1 lists the milestones the collector picked from the git history.

Table 1: Milestones from the git history, all on 2026-10-02.

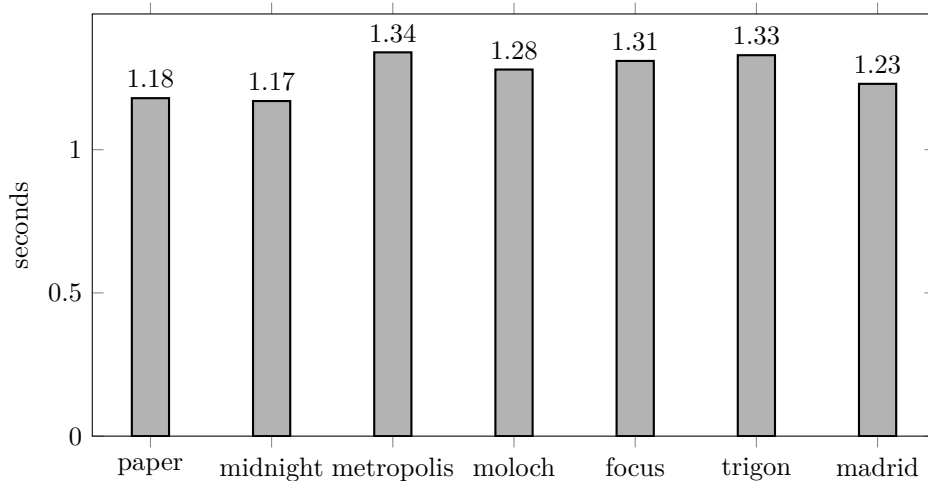
Date	Event
2026-10-02	Project plan and handoff notes
2026-10-02	Server: multipart upload, zip and GitHub intake, collector and agent run, SSE progress
2026-10-02	UI: drop zone, repo link, report or slides, themes, questions, live feed, PDF view
2026-10-02	A <code>.pptx</code> template accepted for slides
2026-10-02	<code>scripts/fill.py</code> measures per-slide body fill
2026-10-02	Revisions can add files, a zip, a repo or notes after the first draft
2026-10-02	UI shows one stage at a time; mobile layout; “Add material” in revise
2026-10-02	Settings and API keys read from a gitignored <code>.env</code>

4 Results

All measurements below were made on my MacBook (Apple Silicon) on 2026-10-03. The test suite, `node --test test.js`, runs 18 tests covering the collector, the template extractor and the agent’s sandbox; all 18 pass, in 0.88 s.

4.1 Compile time

Theme switching only works as a one-click action if a recompile is fast. Figure 3 shows the warm `tectonic main.tex` time for the same 10-slide deck in each of the seven themes, measured on the second run with the TeX cache warm. Figure 4 shows how the time grows when the deck’s frames are repeated one to five times in the Midnight theme: from 1.19 s at 10 slides to 2.04 s at 50.

**Figure 3:** Warm compile time of the same 10-slide deck in each slide theme.

4.2 Slide fill

Early decks came out with large empty areas, so I wrote `scripts/fill.py` to measure the share of each slide’s body area that content fills. Table 2 gives its output on the 10-slide deck in each theme: the mean fill per slide and the lowest single slide. Midnight has the highest mean at 85% and Madrid the lowest at 71%.

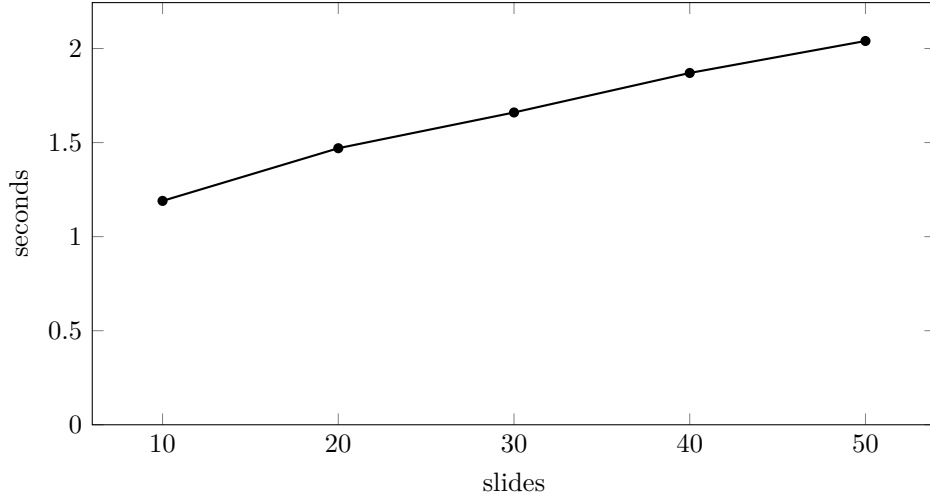


Figure 4: Warm compile time against deck length in the Midnight theme.

Table 2: Slide body fill on the same 10-slide deck, measured by `scripts/fill.py`.

Theme	Mean fill (%)	Lowest slide (%)
paper	78	47
midnight	85	59
metropolis	83	60
moloch	75	51
focus	76	51
trigon	83	51
madrid	71	41

4.3 Time to first PDF

Figure 5 shows the wall time from upload to the first `main.pdf` for 24 local test runs on 2026-10-01 and 2026-10-02. Runs under 30s or over 15 minutes were excluded, since those were copied fixtures or paused sessions. The fastest run took 0.6 minutes and the slowest 12.69 minutes; with a strong model I expect a run to take about 1–3 minutes.

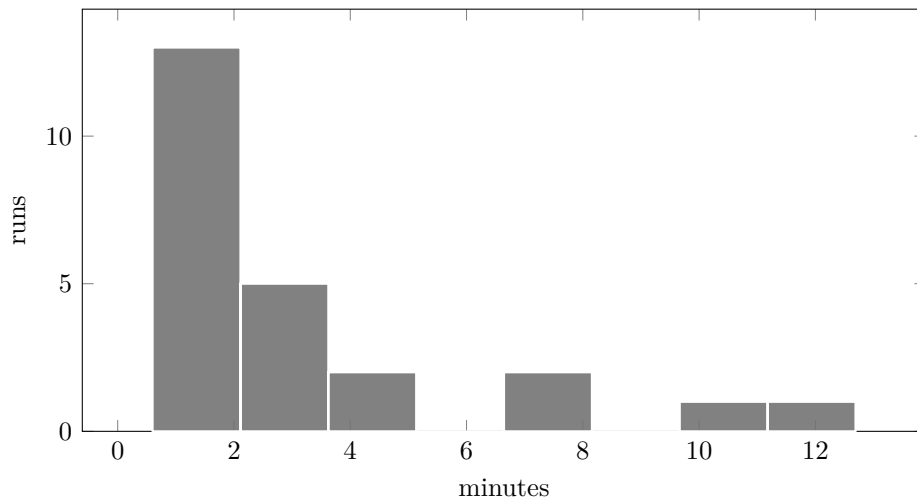


Figure 5: Distribution of minutes from upload to the first PDF over 24 local test runs.

4.4 Small models

With the `ai` engine on an M4 with 16 GB, `qwen3:8b` through Ollama built the sample deck in about 7 minutes. It used real names and features from the README, placed the screenshot and drew a flow and a timeline, but it had text overflow, overlapping timeline dates and no charts. It did not ask questions on its own and sometimes could not get past a compile error.

5 Discussion

The most useful lesson was that a prompt rule loses to the data it is given. The skill once told the agent to use every chart listed in `facts.charts`, and the collector listed charts for commits per hour, most-edited files and commit types, so early reports were full of metrics about my commit habits. Rewording the prompt did not stop it. Removing those series from the chart list in the collector did, and the CSVs are still written for anyone who wants them.

Layout turned out to be a type-scale problem. Slides came out about a third empty, and each theme spaced things differently. Asking the model to “fill the slide” did not help; banning raw font sizes, letting each theme own one type scale, and measuring the result with `scripts/fill.py` instead of judging by eye did.

I only trusted the sandbox after testing it. With a bare `Write` in the allowlist, `claude -p` could write to `../escape.txt`, while `Write(./**)` blocked it. For my own agent loop, `test.js` tries to escape through symlinks, `../` paths and the server’s own files. Server-sent events also dropped silently on idle connections while the agent was thinking; the server now pings every 15 seconds, and the client reconnects, replays the feed without duplicating steps and reattaches to a running job after a page reload.

Smaller models failed in predictable ways: they repeated the same call, stopped before there was a PDF, or got stuck on compile errors inside the `\dr*` macros. Each failure became a guard rail in `agent.js`: at most 80 model steps, the same call three times in a row refused, up to two nudges to compile, and a compile error inside a macro answered with that macro’s usage.

The system has known limits. It runs locally, one job at a time. Template import copies colours, fonts, the logo and rough layout, but not shapes, gradients or animations. 16-bit PNG screenshots are re-encoded with `sips`, which only exists on macOS, and the page loads `pdf.js` and fonts from the network. Uploaded text still reaches an agent that can write files in the job folder, so `devreport` should only be run on trusted material. There is no public hosted version on purpose, since it would run strangers’ jobs on one person’s account.

6 Future work

The next steps are running more than one job at a time and replacing `sips` so that 16-bit PNG screenshots also work on Linux.

Appendix: repository statistics

The repository has 81 commits by one author, Celibistrial, all made on 2026-10-02, with 4644 lines added and 711 removed. The largest source files are `index.html` (1241 lines), `server.js` (633), `collect.js` (453), `test.js` (360), `agent.js` (274) and `pptx.js` (165).